

大语言模型分布式预训练

本节目标

- ◆ 深入了解分布式预训练的任务、损失函数及其关键技术

预训练任务

大语言模型预训练任务

◆ 任务定义

- 给定输入 token 序列 $x_1, x_2, \dots, x_t$ 模型需预测下一个 token x_{t+1} 的概率分布
- **自回归特性**：预测结果会追加到输入序列中，迭代生成后续词元，直至输出终止符（如 `<EOS>`）或达到长度限制

◆ 数据构造

- 文本分词后转为数字序列（如 "你:9, 知道:3"）
- **高效训练技巧：通过移位构造输入 - 标签对**：
 - 输入序列：[9, 3, 6, 4, 2, 1, 5, 0, 0, 0]
 - 标签序列：[3, 6, 4, 2, 1, 5, 0, 0, 0, 0]
 - 模型在单次前向传播中并行计算所有位置的预测损失，提升效率

大语言模型预训练任务

◆ label 构造

- 输入序列：[9, 3, 6, 4, 2, 1, 5, 0, 0, 0]
- 标签序列：[3, 6, 4, 2, 1, 5, 0, 0, 0, 0]

```
# Unpack.
tokens_ = data_b["text"].long()
if "label" in data_b:
    labels = torch.where(
        data_b["label"].long() >= 0,
        data_b["label"].long(),
        torch.zeros_like(data_b["label"].long()),
    )[:, 1:].contiguous()
else:
    labels = tokens_[:, 1:].contiguous()
tokens = tokens_[:, :-1].contiguous()
```

损失函数交叉熵 (Cross Entropy)

◆ 核心原理

- **任务目标**：根据前文预测下一个词元 (Token)，即自回归语言建模 (Causal Language Modeling, CLM)
- **数学形式**：其中 y_i 是真实的下一个词元， $P(y_i)$ 是模型预测的概率分布：

$$L = -\frac{1}{N} \sum_{i=1}^N \log(P(\hat{y}_i | y_{1:i}))$$

1. 移位：预测第 t 个词元时，标签应为第 $t+1$ 个

```
shift_logits = logits[:, :-1, :]
```

移除最后一个词元的预测

```
shift_labels = labels[:, 1:]
```

移除第一个词元的标签

#2. 展平维度

```
shift_logits = shift_logits.contiguous().view(-1, 5) #
```

```
[batch*(seq_len-1), vocab_size]
```

```
shift_labels = shift_labels.contiguous().view(-1) #
```

```
[batch*(seq_len-1)]
```

3. 计算交叉熵损失 (自动忽略标签为 -100 的位置)

```
loss_fn = nn.CrossEntropyLoss(ignore_index=-100)
```

```
loss = loss_fn(shift_logits, shift_labels)
```

分布式训练简介

分布式训练简介

◆ 原因

- ① 模型规模迅速增加，无法装入一个 GPU
- ② 数据集规模迅速增加，单卡训练效率低
- ③ 计算能力越来越强，有条件利用分布式提高效率

分布式训练简介

◆ 模型规模迅速增加

- 2015 年的 ResNet50 有 2000 万的参数
- 2018 年的 BERT-Large 有 3.45 亿的参数
- 2018 年的 GPT-2 有 15 亿的参数
- 2020 年的 GPT-3 有 1750 亿个参数。

模型规模随着时间的推移呈指数级增长。

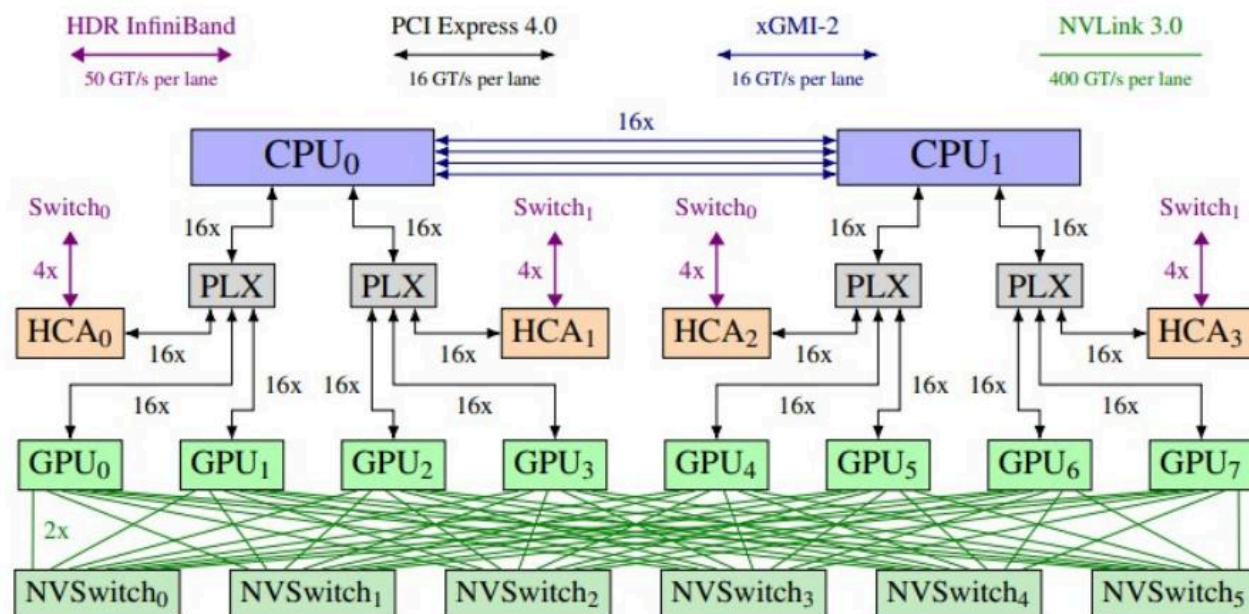
分布式训练简介

◆ 模型规模迅速增加

- 早期的 MNIST 和 CIFAR10 数据集很小然而
- 著名的 ImageNet 数据集数据量迅速增加
- 谷歌未公布的 JFT-300M 数据集，有大约 3 亿张图片，比 ImageNet-1k 数据集大了近 300 倍
- **GPT4 13 万亿 token**

分布式训练简介

◆ 计算能力越来越强



核心并行策略

策略	原理	优势	局限
数据并行	数据分片，每设备持完整模型副本，独立计算梯度后同步平均梯度	实现简单，适合中小模型	通信开销大，显存冗余高
模型并行	模型参数切分到多设备：	突破单卡显存限制	设备依赖性强，调度复杂
- 张量并行	单层参数切分（如矩阵乘行列拆分）	计算效率高（Megatron-LM）	设备间通信频繁
- 流水线并行	模型按层拆分，设备间流水线执行	减少显存占用（DeepSpeed）	存在“流水线气泡”降低利用率
混合并行	结合数据 / 张量 / 流水线并行（如 DeepSpeed 3D 并行）	最大化资源利用率（BLOOM 训	配置复杂度高

关键技术优化

● 显存优化

- **混合精度训练**：FP16/BF16 计算 + FP32 优化器状态，减少显存占用 50% 以上¹
- **ZeRO 优化器 (DeepSpeed)**：
 - ZeRO-1：优化器状态分片 → 显存占用降为 1/4
 - ZeRO-2：梯度分片 → 显存占用降为 1/8
 - ZeRO-3：参数分片 → 支持万亿参数模型
- **梯度检查点**：以计算换显存，存储部分中间结果

● 通信优化

- **集合通信原语**：All-Reduce (梯度聚合)、All-Gather (参数同步) 等高效通信协议
- **计算 - 通信重叠**：在前向传播中预取参数 (如 EDiT 的层间同步)
- **异步训练**：如 A-EDiT 允许异构设备按自身速度训练，减少等待时间

● 稳定性与效率提升

- **伪梯度惩罚 (EDiT)**：抑制因数据分布差异导致的损失尖峰，提升收敛稳定性
- **1-bit Adam (DeepSpeed)**：量化梯度减少通信量，带宽需求降为 1/81

下次预告：大模型分布式预训练之数据并行

分布式训练之数据并行

本节目标

- 深入了解分布式训练中的数据并行中的关键内容

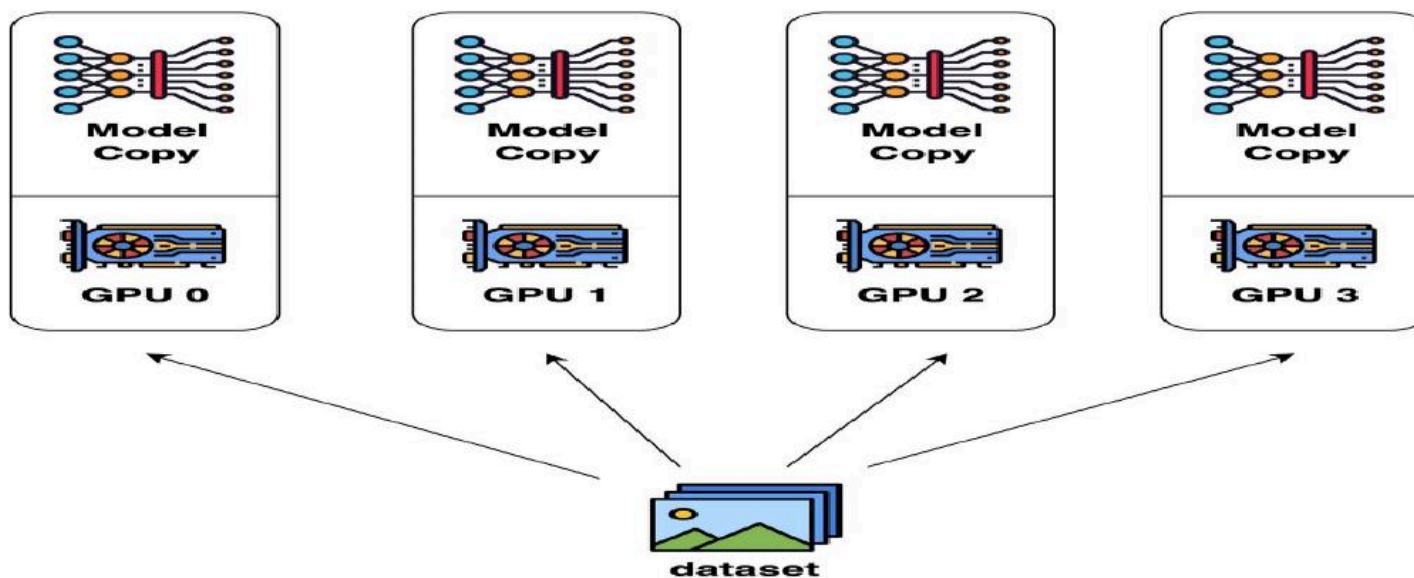
分布式训练之数据并行

□ 数据并行

- ① 很容易理解，数据并行是最**常见**的并行形式
- ② 在数据并行训练中，数据集被分割成几个碎片，每个碎片被分配到一个设备上。这相当于沿批次维度对训练过程进行并行化
- ③ 每个设备将**持有一个完整的模型副本**，并在分配的数据集碎片上进行训练
- ④ 在反向传播之后，模型的梯度将被全部更新，以便在不同设备上的模型参数能够**保持同步**

分布式训练之数据并行

□ 数据并行



分布式训练之数据并行

□ 数据并行 - 增加 GPU ， 提升训练效率

- ① 以常见的 ResNet50 模型使用 32GB V100 卡训练为例。假设训练时单卡最大能支持的 local batch size 为 256 ， 训练一个 step 的耗时为 1 秒， 则单卡训练时的吞吐为 256 imgs/s 。
- ② 如果我们使用 32 张 V100 做数据并行训练， 假设没有损耗， 那么理论上的训练吞吐可达到 $32 \times 256 = 8192$ imgs/ 。
- ③ 实际上由于数据并行时多机多卡的通信消耗等， 实际加速效率会有折扣， 但在加速效率为 0.8 时， 训练吞吐也可达到 $32 \times 256 \times 0.8 = 6554$ imgs/s 。
- ④ 如果使用更多的 GPU ， 并行训练的速度将会更高， 大大减少训练需要的时间。

分布式训练之数据并行

□ ① 输入数据切分

输入数据切分实现上比较简单，一般有两种常用的实现方式：

- 方式一：在每个训练 Epoch 开始前，将整个训练数据集根据并行进程数划分，每个进程只读取自身切分的数据。
- 方式二：数据的读取仅由具体某个进程负责（假设为 rank0）。rank0 在数据读取后同样根据并行进程数将数据切分成多块，再将不同数据块发送到对应进程上。

方式一相对方式二不需要进行数据通信，训练效率更高，飞桨框架中默认的数据并行使用方式一完成数据在不同进程上的切分。

分布式训练之数据并行

□ ② 模型参数同步

数据并行实现的关键问题在于如何保证训练过程中每个进程上模型的参数相同

训练过程的每一个 step 都会更新模型参数，每个进程处理不同的数据会得到不同的 Loss 。由 Loss 计算反向梯度并更新模型参数后，如何保证进程间模型参数正确同步，是数据并行需要解决的最主要问题。根据下面中的梯度更新公式，只要保证以下两点就能解决这个问题：

- ① 每个进程上模型的初始化参数相同
- ② 每个进程上，每次更新时，梯度相同

分布式训练之数据并行

□ ② 模型参数同步

保证每个进程模型参数初始相同有两种常用的实现方法：

方法一：所有进程在参数初始时使用相同的随机种子并以相同的顺序初始化所有参数

方法二：通过一个具体进程初始化全部模型参数，之后由该进程向其他所有进程广播模型参数

分布式训练之数据并行

□ ③ 梯度更新

① 前向计算

每个进程根据自身得到的输入数据独立前向计算，因为输入数据不同每个进程会得到不同的 Loss

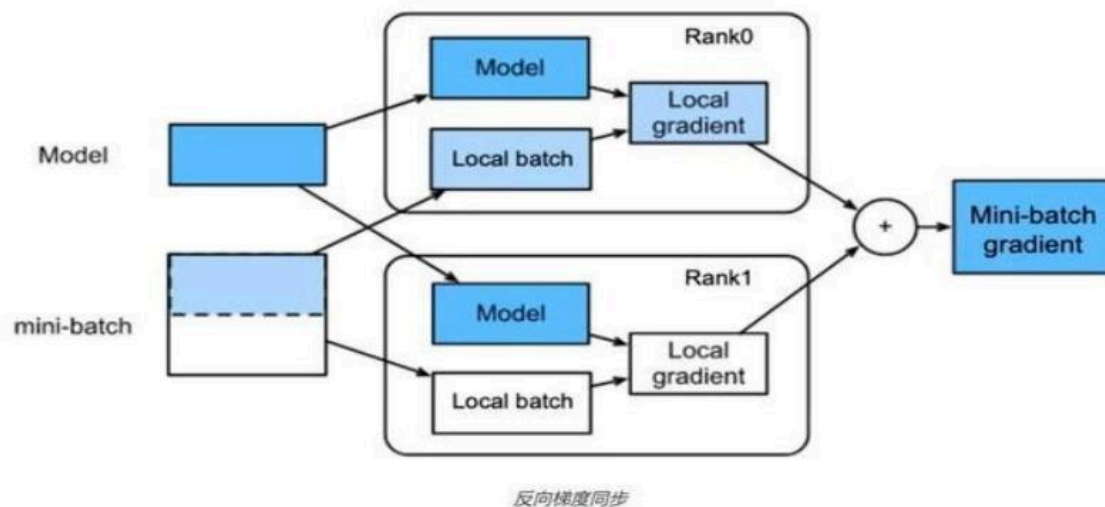
② 反向计算

每个进程根据自身的前向计算独立进行反向计算，因为每个进程上的 Loss 不同，每个进程上在反向中会计算出不同的梯度。更新参数之前，要对所有进程上的梯度进行同步，保证后续更新步骤中每个进程使用相同的全局梯度更新模型参数

分布式训练之数据并行

③ 参数更新

每个进程经过上述步骤后得到相同**全局梯度**，然后各自独立地完成参数更新。因为更新前模型各进程间的参数是相同的，更新中所使用的梯度也是相同的，所以更新后各进程上的参数也是相同的



下次预告：分布式训练之模型并行

分布式训练之模型并行 与异构并行

本节目标

- 深入了解分布式训练中的模型并行与异构系统并行

分布式训练之模型并行

□ 模型并行

- ① 随着技术的发展，业界内训练的模型越来越大，模型朝着**更深和更宽**的方向发展
- ② 以自然语言处理（NLP）领域为例，模型从 Bert 发展到 GPT，模型规模从数亿参数量增加到**数百亿甚至是数千亿**
- ③ 当参数规模为**千亿**时，加载模型参数就需要**数百 GB** 的显存空间，超出单个 GPU 卡的显存容量
- ④ 可以采用模型并行技术，对模型切分，将不同部分加载到不同的显卡上

分布式训练之模型并行

□ 模型并行

流水线并行：

按模型的 layer 层切分到不同设备，即层间并行，称之为**流水线并行**。

张量并行：

将计算图中的层内的参数切分到不同设备，即层内并行，称之为**张量模型并行**。

分布式训练之模型并行

□ 张量并行

张量并行技术需要解决两个问题：

- ① 参数如何切分到不同设备（切分方式）
- ② 切分后，如何保证数学一致性（切分前后，数学等价）

分布式训练之模型并行

□ 张量并行

Transformer 结构主要有：

- ① 嵌入式表示（ Embedding ）层、
- ② 矩阵乘层（ MatMul ）
- ③ 交叉熵 loss 计算层（ CrossEntropy ）

以上**三种类型的组网层**有较大的特性差异，需要分别设计对应的张量模型并行策略，但总体上看核心思想都是利用**分块矩阵**的计算原理，实现其参数切分到不同的设备

分布式训练之模型并行

- 张量并行 -embedding 层处理

分布式训练之模型并行

□ 张量并行 -embedding 层处理

- ① 对于 Embedding 算子，如果总的词表非常大，会导致单卡显存无法容纳 Embedding 层参数。
- ② Embedding 层的参数，按照词的维度切分，即每张卡只存储部分词向量表，然后通过 AllReduce 汇总各个设备上的部分词向量结果，从而得到完整的词向量结果。

当词表数量是 50304，词表表示维度为 5120，类型为 FP32，那么整层参数需要显存大约为 $50304 * 5120 * 4 / 1024 / 1024 = 982\text{MB}$ ，反向梯度同样需要 982MB，仅仅存储就需要将近 2GB。

分布式训练之模型并行

□ 张量并行 -embedding 层处理

embedding 层分布式计算过程：

- ① 将 Embedding 参数沿 word_size 维度，切分为两块，每块大小为 $[\text{word_size}/2, \text{hidden_size}]$ ，分别存储在两个设备上。
- ② 当每张卡查询各自的词表时，如果无法查到，则该词的表示为 0 各自设备查询后得到 $[\text{bz}, \text{hidden_size}]$ 结果张量
- ③ 最后通过 AllReduce_Sum 通信，跨设备求和，得到完整的全量结果

分布式训练之模型并行

□ 张量并行 -embedding 层处理

bz 是 batch size 大小， Embedding 的参数大小为 [word_size, hidden_size] ， 希望得到 [bz, hidden_size] 张量

分布式训练之模型并行

□ 张量并行 - 矩阵乘处理

- ① 对于矩阵乘法 $Y=X*A$ ，其中 X 是维度为 $M \times N$ 的输入矩阵， A 是维度为 $N \times K$ 的参数矩阵， Y 是结果矩阵，维度为 $M \times K$
- ② 如果参数矩阵 A 非常大，甚至**超出单张卡的显存容量**，那么可以把参数矩阵 A 切分到多张卡上，并通过集合通信汇集结果，保证最终结果在数学计算上等价于单卡计算结果

分布式训练之模型并行

□ 张量并行 - 矩阵乘处理

① 参数矩阵 A 按列切块

- 分别将 A_1 , A_2 放置在两张卡上
- 两张卡分别计算 $Y_1 = X * A_1$ 和 $Y_2 = X * A_2$
- AllGather 操作获取其它卡上的计算结果，拼接得到结果矩阵

”

分布式训练之模型并行

□ 张量并行 - 矩阵乘处理

② 参数矩阵 A 按行切块

① 为了满足矩阵乘法规则，
输入矩阵 X 需要按列切分
 $X=[X1 \parallel X2]$

② 将矩阵分块，分别放置在
两张卡上，每张卡分别计
算

$$Y1=X1*A1, \quad Y2=X2*A2$$

③ Allreduce_sum，归约其
他卡上的计算结果，得到
最终的结果矩阵 Y

分布式训练之模型并行

□ 张量并行 - 矩阵乘处理

□ Transformer 中的 FFN 层

分布式训练之模型并行

□ 张量并行 - 矩阵乘处理

- Transformer 中的 FFN 结构均包含两层全连接（FC）层，即存在两个矩阵乘；
- 对第一个 FC 层的参数矩阵按列切块，对第二个 FC 层参数矩阵按行切块；
- 第一个 FC 层的输出恰好满足第二个 FC 层数据输入要求（按列切分），可以省去第一个 FC 层后的 AllGather 通信操作

分布式训练之模型并行

□ 张量并行 - 交叉熵 Loss 计算

- ① 首先是如何计算 softmax 值，如下公式，其中 M 表示张量模型并行的设备号， x 是 logits，分布在多张卡上

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}} = \frac{e^{x_i - x_{\max}}}{\sum_j e^{x_j - x_{\max}}} = \frac{e^{x_i - x_{\max}}}{\sum_M \sum_k e^{x_k - x_{\max}}}$$

$x_{\max} = \max_M (\max_k (x_k))$

- ② 同时，对标签 target 按类别切分，每个设备得到部分 loss
- ③ 最后，在进行一次集合通信，得到全量的 loss

并行

$$\text{softmax}(x_i) = \frac{e^{x_i - x_{\max}}}{\sum_k e^{x_k - x_{\max}}}$$

$$\text{loss} = -\log p(\hat{y}_t | y_{\square}) = -\log \frac{e^{x_t - x_{\max}}}{\sum_k e^{x_k - x_{\max}}}$$

$$-\log(\sum_k e^{x_k - x_{\max}}) = -\log(e^{x_t - x_{\max}})$$

$$-(x_t - x_{\max}) = -\log(\sum_k e^{x_k - x_{\max}})$$

分布式训练之流水线并行

分布式训练之流水线并行

□ 流水线并行

- ① 张量模型并行可以把 shape 较大的参数（Tensor）切分到多个卡，从而有效减小在每个卡上的参数量
- ② 但是，采用这种方式单机 8 卡（V100，32GB）最多训练 10B 左右的 Dense 模型
- ③ 而且，由于其通信和计算不能重叠的特点一般不适合多机之间使用
- ④ 更大的模型需要从层（Layer）级别进行进一步的图切分以减少单卡存储的容量，同时隐藏卡之间的通信时间为此业界提出了**流水线并行**

分布式训练之流水线并行

□ 流水线并行

- ① 流水线并行的**核心思想**是，模型按层分割成若干块，每块都交给一个设备。
- ② 在前向传递过程中，每个设备将中间的激活传递给下一个阶段。
- ③ 在后向传递过程中，每个设备将输入张量的梯度传回给前一个流水线阶段。
- ④ 这允许设备同时进行计算，并增加了训练的吞吐量。流水线并行训练的一个缺点是，会有一些设备参与计算的**冒泡时间**，导致计算资源的浪费。

分布式训练之流水线并行

□ 流水线并行

分布式训练之异构系统并行

分布式训练之异构系统并行

□ 异构系统并行

□ 与 GPU 相比，CPU 的内存要大得多。

- 在一个典型的服务器上，CPU 可以轻松拥有几百 GB 的内存，而每个 GPU 通常只有 16 或 32GB 的内存。为什么 CPU 内存没有被用于分布式训练？

□ 可以依靠 CPU 甚至是 NVMe 磁盘来训练大型模型

- 主要的想法是，在不使用张量时，将其卸载回 CPU 内存或 NVMe 磁盘。通过使用异构系统架构，有可能在一台机器上容纳一个巨大的模型。

分布式训练之异构系统并行

□ 异构系统并行

下次预告： 分布式训练框架 `deepspeed`

分布式训练框架介绍

本节目标

- 深入了解模型训练时的显存占用情况
- 分布式训练软件框架 **deepspeed** 及其优化策略介绍

大模型训练显存占用分析

显存占用分析

① **Model States** 模型本身相关且必须存储的参数：

- ☐ Parameters : 模型参数
- ☐ Gradients : 模型梯度
- ☐ Optimizer States : Adam 中 momentum 和 variance

② **Residual States** 非模型必须，训练过程中产生的参数：

- ☐ Activation : 激活值
- ☐ Temporary Buffers : 临时存储
- ☐ Unusable Fragmented Memory : 碎片化存储空间

显存占用分析

- **Model States :**

- ① Parameters (half) : 2 bytes
- ② Gradients (half) : 2 bytes

- **Optimizer States :**

- ③ Master Weight (fp32) : 4 bytes
- ④ Adam momentum (fp32) : 4 bytes
- ⑤ Adam Variance (fp32) : 4 bytes

显存占用分析

假设模型参数量 Ψ ，使用 **Adam** 优化器混合精度训练：

- ① 模型参数和梯度 float16 ，显存消耗为 $2\Psi + 2\Psi$;
- ② **Adam** 维护 **float32** 模型副本，消耗 4Ψ ;
- ③ Adam 辅助变量 fp32 momentum + fp32 variance ，显存消耗 $4\Psi + 4\Psi$;
- ④ 模型消耗 $2\Psi + 2\Psi = 4\Psi$ ，Adam 消耗 $4\Psi + 4\Psi + 4\Psi = 12\Psi$ ，总消耗 **$4\Psi + 12\Psi = 16\Psi$**

优化器显存占用表示 $K\Psi$ （不同的优化器不同），则混合精度训练显存占用 $4\Psi + K\Psi$

分布式训练框架

分布式训练并行意义

分布式训练并行意义

分布式并行意义

deepspeed

- 支持大模型全流程

deepspeed

□ deepspeed 特性

Training : 为大模型训练供 ZeRO 、 3D-Parallelism 、 DeepSpeed-MoE 、 ZeRO-Infinity 等特性;

Inference : 提供 Tensor 、 Pipeline 、 Expert 等并行特性与推理内核、通信优化和异构内存特性结合;

Compression : 提供 ZeroQuant 和 XTC 等 SoTA 在压缩方面的创新;

4Science : 兼容性, 效率等;

分布式训练及 **deepspeed**

□ **deepspeed** 特性

分布式训练及 deepspeed

□ 软件架构

- ① RunTime：核心运行时组件，负责管理、执行和优化性能。包括数据、模型、并行优化、微调、故障检测以及 CheckPoint 保存和加载等任务。
- ② Ops：底层内核组件，使用 C++ 和 CUDA 实现。优化计算和通信，提供底层操作；
- ③ APIs：配置参数在 ds_config.json 中，通过 API 接口可以调用 DeepSpeed 训练 / 推理模型；

分布式训练及 **deepspeed**

□ 软件架构

分布式训练及 **deepspeed**

□ 混合精度

分布式训练及 deepspeed

□ ZERO 并行

ZeRO Redundancy Optimizer ，一系列显存优化方法的统称：

- ① **ZeRO-DP** （ Data Parallel ）： ZeRO1/2/3
- ② **ZeRO-R** （ Reduce ）： Activation Checkpointing 、
Memory Defragmentation
- ③ **ZeRO-Offload** ： Offload Strategy && Offload Schedule
- ④ **ZeRO-Infinity** ： Breaking the GPU Memory Wall for
Extreme Scale Deep Learning

分布式训练及 deepspeed

□ ZERO-DP

① Optimizer state partitioning (ZeRO stage 1) :

只对 optimizer 状态进行切分, 占用内存原始 1/4

② Gradient partitioning (ZeRO stage 2) :

对 optimizer 和 grad 进行切分, 占用内存原始 1/8

③ Parameter partitioning (ZeRO stage 3) :

对 optimizer、grad 和模型参数进行切分, 内存减少与数据并行度和复杂度成线性关系, 同时通信容量是数据并行性的 1.5 倍

分布式训练及 deepspeed

□ ZERO-DP

ZERO-stage1

zero-1

□ zero stage1 原理

zero-1

□ zero stage1 原理

- ① 从 optimizer state 开始， **batch** 数据分成 **N** 份，每块 GPU 一份
- ② 执行 1 Step forward & backward 后，各得一份梯度 G_n
- ③ 对**梯度 G_n** 执行 All-Reduce，得到完整梯度 G ，单 GPU 通讯量 2Φ
- ④ 得到完整梯度 G ，对权重 W 更新，权重 W 更新由 optimizer states 和 grad 共同决定。每块 GPU 有部分 W ，对 W 执行 All-Gather，从其他 GPU 上更新好的部分 W ，产生单 GPU 通讯量 Φ

zero-1

□ zero stage1 原理

- ① 从 optimizer state 开始, **batch** 数据分成 **N** 份, 每块 GPU 一份

zero-1

□ zero stage1 原理

- ② 执行 1 Step forward & backward 后，各得一份梯度 G_n

zero-1

□ zero stage1 原理

- ③ 对梯度 G_n 执行 All-Reduce , 得到完整梯度 G , 单 GPU 通讯量 2Φ
- ④ 得到完整梯度 G , 对权重 W 更新, 而权重 W 更新由 optimizer states 和 grad 共同决定。每块 GPU 有部分 W , 对 W 执行 All-Gather , 从其他 GPU 上更新好的部分 W , 产生单 GPU 通讯量 Φ

zero-1

□ zero stage1 原理

- ⑤ 对梯度 G_n 执行 scatter-reduce , 各 GPU 维护 Optimizer 更新对应 w_n
- ⑥ 再对权重 w_n 执行 all-gather 使得每 GPU 都有更新后完整 W , 最终通讯量为 2Φ

zero-1

□ zero stage1 原理

① 在 Pos 阶段优化器状态划分：

- 根据 DP 维度 $\#d$ 将 Adam 优化器状态划分为 $\#d$ 等份；
- 每 NPU 需要存储和更新总优化器状态的 $1/\#d$ ，并更新 $1/\#d$ 参数
- 每训练 Step 末尾，使用 all-gather 获得整个参数的更新完整的权重 w_n

② 显存分析：

- 显存从 $4\Psi + K\Psi$ 降低到 $4\Psi + K\Psi/\#d$
- 当 $\#d$ 很大时，显存占用接近于 4Ψ ，带来 4 倍显存节约。

zero-stage2

zero-stage2

□ 梯度切分

zero-stage2

□ 梯度切分

- ① 从 optimizer state 开始, batch 数据分成 N 份, 每块 GPU 一份
- ② 执行 One Step forward & backward 后, 各得一份梯度 G_n

zero-stage2

□ 梯度切分

对梯度 G_n 执行 Reduce-Scatter，保证每个 GPU 所维持的梯度 G_n 是聚合梯度：

- ① 对 GPU_1 负责维护梯度 G_1 ，其他 GPU 只需要把 G_1 对应位置梯度发给 GPU_1 即可
- ② 汇总完毕后白色块对 GPU_1 不用从显存移除，单卡通讯量 Φ

zero-stage2

□ 梯度切分

- ③ 每 GPU O_n 和梯度 G_n 更新相应 w_n ，即每块 GPU 维护独立的权重 w_n
- ④ 最后对权重 w_n 执行 All-Gather，将其他 GPU 的 w_n 同步一份完整 w ，单卡通讯量 Φ

zero-stage2

□ 梯度切分

- ① **batch** 数据分成 **N** 份，每块 GPU 一份
- ② 执行 **One Step forward & backward** 后，各得一份梯度 G_n
- ③ 对梯度 G_n 执行 **Reduce-Scatter**，保证每个 GPU 所维持的梯度 G_n 是聚合梯度：
例如，对 GPU1 负责维护梯度 G_1 ，其他 GPU 只需要把 G_1 对应位置梯度发给 GPU1 即可；汇总完毕后白色块对 GPU1 不用从显存移除，单卡通讯量 Φ
- ④ 每 GPU O_n 和梯度 G_n 更新相应 w_n ，每块 GPU 维护独立的权重 w_n
- ⑤ 最后对权重 w_n 执行 All-Gather，将其他 GPU 的 w_n 同步一份完整 w ，单卡通讯量 Φ

zero-stage2

□ 梯度切分

① 在 p_g 阶段优化器状态 + 梯度划分:

- 梯度跟优化器强相关，因此优化器可以更新其独立的梯度参数
- 重点是更新梯度参数时候使用 Reduce-Scatter，梯度参数更新后马上释放，显存从 $2\Psi \rightarrow 2\Psi/d$
- 具体实现：实现过程中使用分桶 Bucket，将梯度分到 Bucket 中并在桶上进行 Reduce-Scatter

zero-stage2

□ 梯度切分

② 显存分析：

- 移除梯度和优化器状态冗余，将显存从 $4\Psi + K\Psi$ 降低到 $2\Psi + K\Psi/d$
- 当 d 很大时，显存占用接近于 2Ψ ，带来 8 倍显存节约

ZERO-stage3

ZERO-stage3

□ 权重划分

ZERO-stage3

□ 模型并行

- ① 将 batch 数据分成 N 份，每块 GPU 一份
- ② Forward 计算，对 w_n 执行 All-Gather 取回分布在各 GPU 上的权重 w_n 得到完整 W ，并把不属于自身的权重 w_{others} 抛弃 -> 单卡通讯量 Φ
- ③ Backward 计算，对 w_n 执行 All-Gather，取回完整 W ，并把不属于自身权重 w_{others} 抛弃 -> 单卡通讯量 Φ
- ④ backward 后得到各自梯度 G_n ，对 G_n 执行 Reduce-Scatter，从其他 GPU 聚合自身维护的梯度 G_n 。聚合操作结束后，立刻把不是自己维护的 G 抛弃 -> 单卡通讯量 Φ
- ⑤ 每 GPU 只保存其权重参数 w_n ，由于只维护部分参数

ZERO-stage3

□ Stage3

- ① Forward 计算，对 w_n 执行 All-Gather 取回分布在各 GPU 上的权重 w_n 得到完整 W ，并把不属于自身的权重 w_{others} 抛弃 -> 单卡通讯量 Φ
- ② Backward 计算，对 w_n 执行 All-Gather，取回完整 W ，并把不属于自身权重 w_{others} 抛弃 -> 单卡通讯量 Φ

ZERO-stage3

□ Stage3

- ③ backward 后得到各自梯度 G_n ，对 G_n 执行 Reduce-Scatter，从其他 GPU 聚合自身维护的梯度 G_n 。聚合操作结束后，立刻把不是自己维护的 G 抛弃 -> 单卡通讯量 Φ
- ④ 每 GPU 只保存其权重参数 w_n ，
由于只维护部分参数 w_n ，
因此无需对 w_n 执行 All-Reduce

ZERO-stage3

□ 权重划分

在 P_p 阶段优化器状态 + 梯度 + 权重划分:

- ① 拆分到 forward & backward 过程，通过 broadcast 从其他 GPU 中获取参数
- ② 通过增加通信开销，减少每张 GPU 中的显存占用，以通信换显存，使得显存占用与 d 成正比

ZERO-stage3

□ 权重划分

显存分析：

- ① 移除梯度、优化器状态、权重冗余，将显存从 $4\Psi + K\Psi$ 降低到 $(4\Psi + K\Psi)/d$
- ② 带来 DP 增加 1.5X 单卡通讯量

ZERO-offload

ZERO-offload

□ OFFLOAD 原理

- ① 把占用显存多的部分卸载 offload 到 CPU 上，计算和激活值部分放到 GPU 上，这样比起跨机，更能节省内存，也能减少跨机跨通信域通讯压力！
- ② 部分 GPU 计算和存储 offload 到 CPU&&DDR，涉及 **CPU2GPU** 间通信增加，如何避免 **host2device** 通信成为瓶颈？
- ③ **GPU** 计算效率相比于 CPU 是数量级上的优，如何避免 CPU 参与过多计算，计算负载尽可能在 GPU？

ZERO-offload

□ OFFLOAD 原理 - 计算量

- ① **高计算**: forward && backward
计算量高, 相关的权重参数计算
& 激活值计算仍然在 GPU
- ② **低计算**: update 部分计算量低以
通信为主, 且需要的显存较大, 放
入 CPU&DDR。相关部分
Optimizer States (fp32)
Update 和 Gradients (fp16)
Update 等。

deepspeed 训练实战

大模型预训练实战

目录

- ◆ 数据预处理
- ◆ 模型选型与配置
- ◆ 预训练框架适配与代码带读
- ◆ 大模型预训练实战

数据预处理

数据准备

◆ chatGPT

数据处理

数据处理代码解读

分布式训练简介

分布式训练简介

◆ 原因

- ① 模型规模迅速增加，无法装入一个 GPU
- ② 数据集规模迅速增加，单卡训练效率低
- ③ 计算能力越来越强，有条件利用分布式提高效率

分布式训练简介

◆ 模型规模迅速增加

- 2015 年的 ResNet50 有 2000 万的参数
- 2018 年的 BERT-Large 有 3.45 亿的参数
- 2018 年的 GPT-2 有 15 亿的参数
- 2020 年的 GPT-3 有 1750 亿个参数。

模型规模随着时间的推移呈指数级增长。

分布式训练简介

◆ 模型规模迅速增加

- 早期的 MNIST 和 CIFAR10 数据集很小然而
- 著名的 ImageNet 数据集数据量迅速增加
- 谷歌未公布的 JFT-300M 数据集，有大约 3 亿张图片，比 ImageNet-1k 数据集大了近 300 倍
- **GPT4 13 万亿 token**

分布式训练简介

◆ 计算能力越来越强

分布式训练简介

◆ 集合通信原语介绍

分布式训练简介

◆ 集合通信原语介绍

分布式训练之数据并行

分布式训练之数据并行

◆ 数据并行

- ① 他很简单，数据并行是最常见的并行形式
- ② 在数据并行训练中，数据集被分割成几个碎片，每个碎片被分配到一个设备上。这相当于沿批次维度对训练过程进行并行化
- ③ 每个设备将**持有一个完整的模型副本**，并在分配的数据集碎片上进行训练
- ④ 在反向传播之后，模型的梯度将被全部更新，以便在不同设备上的模型参数能够**保持同步**

分布式训练之数据并行

◆ 数据并行

分布式训练之数据并行

◆ 数据并行 - 增加 GPU ，提升训练效率

- ① 以常见的 ResNet50 模型使用 32GB V100 卡训练为例。假设训练时单卡最大能支持的 local batch size 为 256 ，训练一个 step 的耗时为 1 秒，则单卡训练时的吞吐为 **256 imgs/s** 。
- ② 如果我们使用 32 张 V100 做数据并行训练，假设没有损耗，那么理论上的训练吞吐可达到 $32 \times 256 = 8192 \text{ imgs/}$ 。
- ③ 实际上由于数据并行时多机多卡的通信消耗等，实际加速效率会有折扣，但在加速效率为 0.8 时，训练吞吐也可达到 $32 \times 256 \times 0.8 =$ **6554 imgs/s** 。
- ④ 如果使用更多的 GPU ，并行训练的速度将会更高，大大减少训练需要的时间。

分布式训练之数据并行

◆ ① 输入数据切分

输入数据切分实现上比较简单，一般有两种常用的实现方式：

- 方式一：在每个训练 Epoch 开始前，将整个训练数据集根据并行进程数划分，每个进程只读取自身切分的数据。
- 方式二：数据的读取仅由具体某个进程负责（假设为 rank0）。rank0 在数据读取后同样根据并行进程数将数据切分成多块，再将不同数据块发送到对应进程上。

方式一相对方式二不需要进行数据通信，训练效率更高，飞桨框架中默认的数据并行使用方式一完成数据在不同进程上的切分。

分布式训练之数据并行

◆ ② 模型参数同步

数据并行实现的关键问题在于如何保证训练过程中每个进程上模型的参数相同。

训练过程的每一个 step 都会更新模型参数，每个进程处理不同的数据会得到不同的 Loss。由 Loss 计算反向梯度并更新模型参数后，如何保证进程间模型参数正确同步，是数据并行需要解决的最主要问题。根据下面中的梯度更新公式，只要保证以下两点就能解决这个问题：

- ① 每个进程上模型的初始化参数相同
- ② 每个进程上，每次更新时，梯度相同

分布式训练之数据并行

◆ ② 模型参数同步

保证每个进程模型参数初始相同有两种常用的实现方法：

方法一：所有进程在参数初始时使用相同的随机种子并以相同的顺序初始化所有参数。

方法二：通过一个具体进程初始化全部模型参数，之后由该进程向其他所有进程广播模型参数。

分布式训练之数据并行

◆ ③ 梯度更新

① 前向计算

每个进程根据自身得到的输入数据独立前向计算，因为输入数据不同每个进程会得到不同的 Loss。

② 反向计算

每个进程根据自身的前向计算独立进行反向计算，因为每个进程上的 Loss 不同，每个进程上在反向中会计算出不同的梯度。更新参数之前，要对所有进程上的梯度进行同步，保证后续更新步骤中每个进程使用相同的全局梯度更新模型参数。

分布式训练之数据并行

◆ ③ 梯度更新

③ 参数更新

每个进程经过上述步骤后得到相同全局梯度，然后各自独立地完成参数更新。因为更新前模型各进程间的参数是相同的，更新中所使用的梯度也是相同的，所以更新后各进程上的参数也是相同的。

分布式训练之模型并行

分布式训练之模型并行

◆ 模型并行

- ① 随着技术的发展，业界内训练的模型越来越大，模型朝着**更深和更宽**的方向发展。
- ② 以自然语言处理（ NLP ）领域为例，模型从 Bert 发展到 GPT ，模型规模从数亿参数量增加到**数百亿甚至是数千亿**
- ③ 当参数规模为**千亿**时，加载模型参数就需要**数百 GB** 的显存空间，超出单个 GPU 卡的显存容量。
- ④ 可以采用模型并行技术，对模型切分，将不同部分加载到不同的显卡上。

分布式训练之模型并行

◆ 模型并行

流水线并行：

按模型的 layer 层切分到不同设备，即层间并行，称之为**流水线并行**。

张量并行：

将计算图中的层内的参数切分到不同设备，即层内并行，称之为**张量模型并行**。

分布式训练之模型并行

◆ 张量并行

张量并行技术需要解决两个问题：

- ① 参数如何切分到不同设备（切分方式）
- ② 切分后，如何保证数学一致性（切分前后，数学等价）

分布式训练之模型并行

◆ 张量并行

Transformer 结构主要有：

- ① 嵌入式表示 (Embedding) 层、
- ② 矩阵乘层 (MatMul)
- ③ 交叉熵 loss 计算层 (CrossEntropy)

以上**三种类型的组网层**有较大的特性差异，需要分别设计对应的张量模型并行策略，但总体上看核心思想都是利用**分块矩阵**的计算原理，实现其参数切分到不同的设备

分布式训练之模型并行

◆ 张量并行 -embedding 层处理

分布式训练之模型并行

◆ 张量并行 -embedding 层处理

- ① 对于 Embedding 算子，如果总的词表非常大，会导致单卡显存无法容纳 Embedding 层参数。
- ② Embedding 层的参数，按照词的维度切分，即每张卡只存储部分词向量表，然后通过 **AllReduce** 汇总各个设备上的部分词向量结果，从而得到完整的词向量结果。

当词表数量是 50304，词表表示维度为 5120，类型为 FP32，那么整层参数需要显存大约为 $50304 * 5120 * 4 / 1024 / 1024 = 982\text{MB}$ ，反向梯度同样需要 982MB，仅仅存储就需要将近 **2GB**。

分布式训练之模型并行

◆ 张量并行 -embedding 层处理

embedding 层分布式计算过程：

- ① 将 Embedding 参数沿 word_size 维度，切分为两块，每块大小为 $[\text{word_size}/2, \text{hidden_size}]$ ，分别存储在两个设备上。
- ② 当每张卡查询各自的词表时，如果无法查到，则该词的表示为 0 各自设备查询后得到 $[\text{bz}, \text{hidden_size}]$ 结果张量
- ③ 最后通过 AllReduce_Sum 通信，跨设备求和，得到完整的全量结果

分布式训练之模型并行

◆ 张量并行 -embedding 层处理

bz 是 batch size 大小，Embedding 的参数大小为 [word_size, hidden_size]，希望得到 [bz, hidden_size] 张量

分布式训练之模型并行

◆ 张量并行 - 矩阵乘处理

- ① 对于矩阵乘法 $Y=X*A$ ，其中 X 是维度为 $M \times N$ 的输入矩阵， A 是维度为 $N \times K$ 的参数矩阵， Y 是结果矩阵，维度为 $M \times K$ 。
- ② 如果参数矩阵 A 非常大，甚至超出单张卡的显存容量，那么可以把参数矩阵 A 切分到多张卡上，并通过集合通信汇集结果，保证最终结果在数学计算上等价于单卡计算结果。

分布式训练之模型并行

◆ 张量并行 - 矩阵乘处理

① 参数矩阵 A 按列切块

- 分别将 A_1 , A_2 放置在两张卡上
- 两张卡分别计算 $Y_1 = X * A_1$ 和 $Y_2 = X * A_2$
- AllGather 操作获取其它卡上的计算结果，拼接得到结果矩阵 Y

分布式训练之模型并行

◆ 张量并行 - 矩阵乘处理

② 参数矩阵 A 按行切块

- ① 为了满足矩阵乘法规则，输入矩阵 X 需要按列切分 $X=[X1 \mid X2]$
- ② 将矩阵分块，分别放置在两张卡上，每张卡分别计算
 $Y1=X1*A1$ ， $Y2=X2*A2$
- ③ Allreduce_sum ，归约其他卡上的计算结果，得到最终的结果矩阵 Y

分布式训练之模型并行

◆ 张量并行 - 矩阵乘处理

- Transformer 中的 FFN 层

分布式训练之模型并行

◆ 张量并行 - 矩阵乘处理

- Transformer 中的 FFN 结构均包含两层全连接 (FC) 层，即存在两个矩阵乘；
- 对第一个 FC 层的参数矩阵按列切块，对第二个 FC 层参数矩阵按行切块；
- 第一个 FC 层的输出恰好满足第二个 FC 层数据输入要求 (按列切分)，可以省去第一个 FC 层后的 AllGather 通信操作

分布式训练之模型并行

◆ 张量并行 - 交叉熵 Loss 计算

- ① 首先是如何计算 softmax 值，如下公式，其中 M 表示张量模型并行的设备号，x 是 logits，分布在多张卡上

$$x_{\max} = \max_M (\max_k (x_k))$$
$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}} = \frac{e^{x_i - x_{\max}}}{\sum_j e^{x_j - x_{\max}}} = \sum_M \sum_k \frac{e^{x_i - x_{\max}}}{e^{x_k - x_{\max}}}$$

- ② 同时，对标签 target 按类别切分，每个设备得到部分 loss
- ③ 最后，在进行一次集合通信，得到全量的 loss。

分布式训练之模型并行

◆ 张量并行 -

分布式训练之流水线并行

分布式训练之流水线并行

◆ 流水线并行

- ① 张量模型并行可以把 shape 较大的参数 (Tensor) 切分到多个卡，从而有效减小在每个卡上的参数量
- ② 但是，采用这种方式单机 8 卡 (V100 , 32GB) 最多训练 10B 左右的 Dense 模型
- ③ 而且，由于其通信和计算不能重叠的特点一般不适合多机之间使用
- ④ 更大的模型需要从层 (Layer) 级别进行进一步的图切分以减少单卡存储的容量，同时隐藏卡之间的通信时间为此业界提出了**流水线并行**

分布式训练之流水线并行

◆ 流水线并行

- ① 流水线并行的核心思想是，模型按层分割成若干块，每块都交给一个设备。
- ② 在前向传递过程中，每个设备将中间的激活传递给下一个阶段。
- ③ 在后向传递过程中，每个设备将输入张量的梯度传回给前一个流水线阶段。
- ④ 这允许设备同时进行计算，并增加了训练的吞吐量。流水线并行训练的一个缺点是，会有一些设备参与计算的冒泡时间，导致计算资源的浪费。

分布式训练之流水线并行

◆ 流水线并行

分布式训练之流水线并行

分布式训练之异构系统并行

◆ 异构系统并行

- 与 GPU 相比，CPU 的内存要大得多。

- 在一个典型的服务器上，CPU 可以轻松拥有几百 GB 的内存，而每个 GPU 通常只有 16 或 32GB 的内存。为什么 CPU 内存没有被用于分布式训练？

- 可以依靠 CPU 甚至是 NVMe 磁盘来训练大型模型

- 主要的想法是，在不使用张量时，将其卸载回 CPU 内存或 NVMe 磁盘。通过使用异构系统架构，有可能在一台机器上容纳一个巨大的模型。

分布式训练之异构系统并行

◆ 异构系统并行

下次预告：分布式训练框架 `deepspeed`